

Computergrafik

PD Prof. Dr. rer. nat.
Marco Block-Berlitz

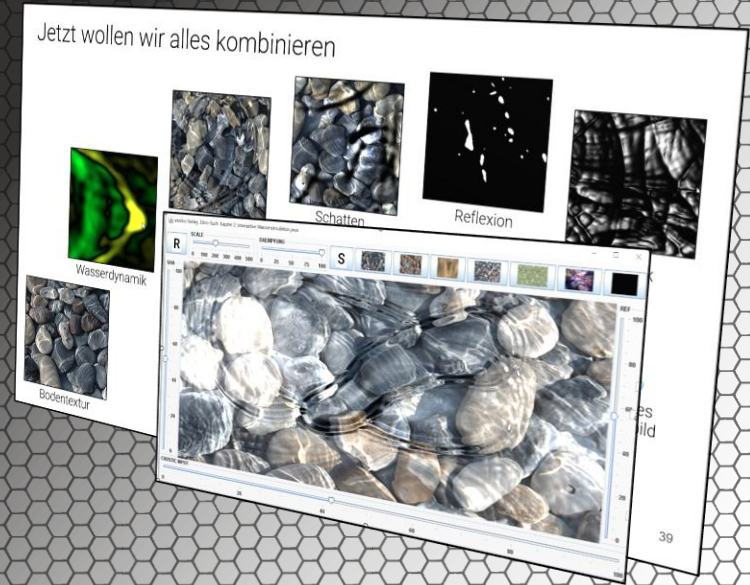
Sommersemester 2026



Entwicklung einer experimentellen Wassersimulation

„Das Wasser nimmt jede Form an, in die man es gießt, und doch bleibt es sich selbst treu.“ (sinngemäß nach Katsushika Hokusai)

PD Prof. Dr. Marco Block-Berlitz



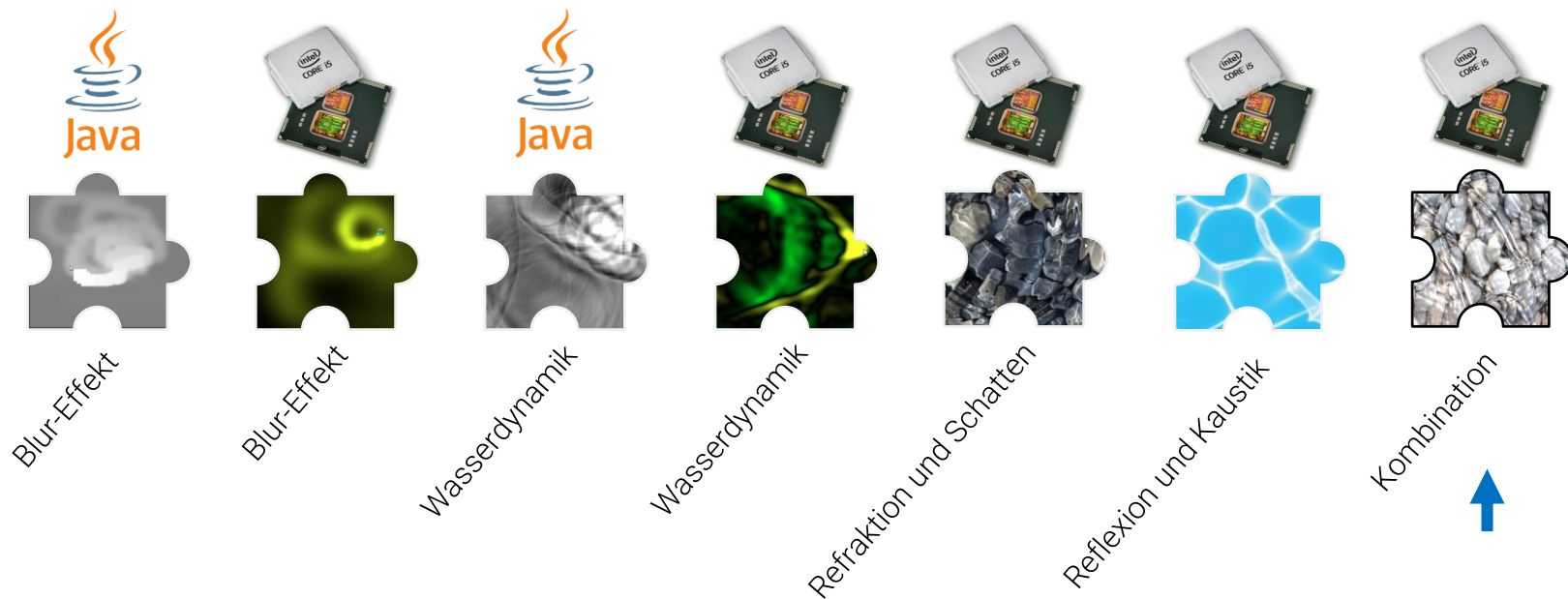
Ziele und Inhalt

- Wasser als Summe seiner Einzeleffekte
- Vielfältige Reflexionen
- Wasser ist nicht gleich Wasser
- Unterschiedliche Wasserfarben
- Automatisierte Korrektur der Wasserfarbe
- Wassersimulation als Kombination der Einzeleffekte
- Projektstufe 7 (Experimentelle Wassersimulation)
- Fragment-Shader Wassersimulation
- Animaggo-Preis
- Vertex-Shader
- Mesh-Generierung und Texturverknüpfung
- Shaderstrategie
- Vertex-Shader Mesh-Verbiegung
- Fragment-Shader Mesh-Verbiegung
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.

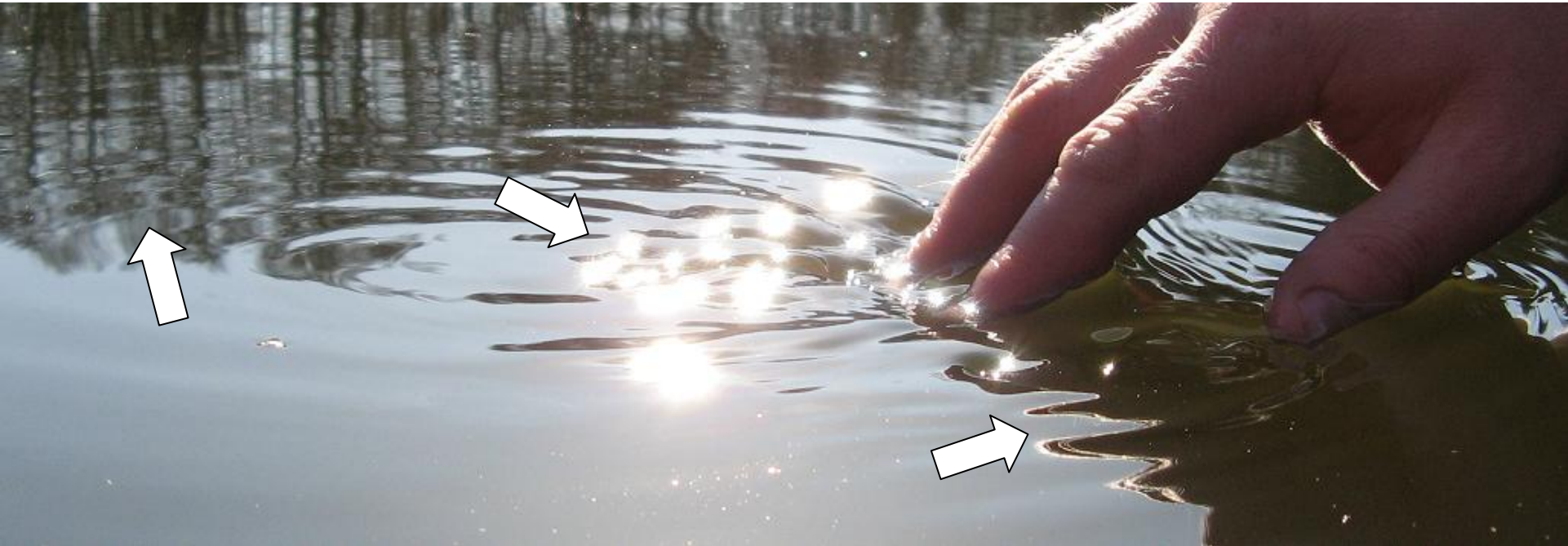




Wasser als **Summe seiner Einzeleffekte**

Reflexionen an der Wasseroberfläche

Je intensiver wir uns mit dem Medium Wasser auseinandersetzen, desto mehr interessante Effekte können wir entdecken.



Deutschland, Zwischenahner Meer

Wir werden schnell feststellen, dass Wasser nicht gleich Wasser ist! So unterscheiden sich die Gewässer durch die Bestandteile auch in der **Farbe**.

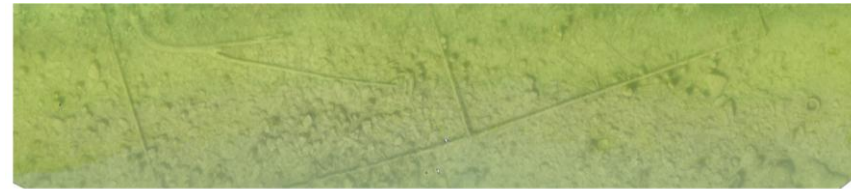
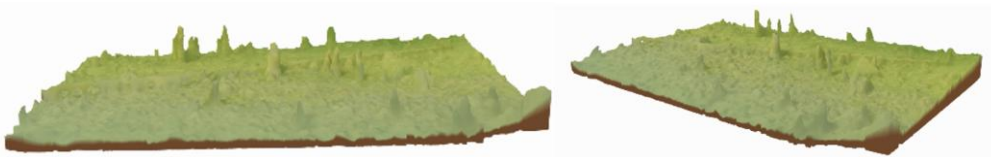
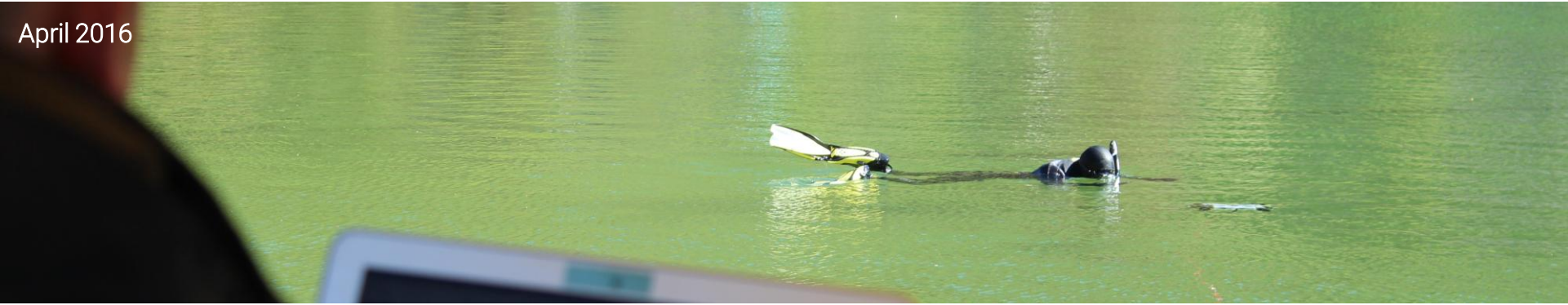
März 2016



Österreich, Mondsee

Wir werden schnell feststellen, dass Wasser nicht gleich Wasser ist! So unterscheiden sich die Gewässer durch die Bestandteile auch in der Farbe.

April 2016

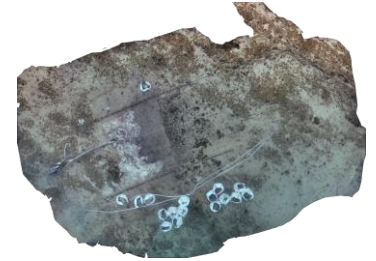
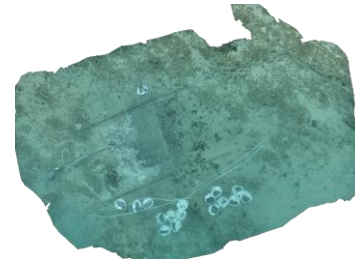
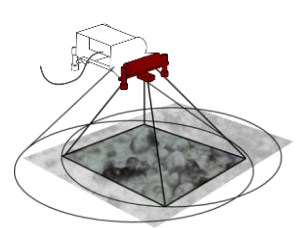


[1] Block M., Dworsky C., Löw C., Seidl da Fonseca H., Gehmlich B., Wittchen D., Görsch N., Suchowska P., Ducke B.: Archaeonautic: Low-cost and efficient 3D Videogrammetry meets Pile Dwellings at "See am Mondsee" in Austria, In Journal of Studies in Digital Heritage. Indiana University. 21st Conference on Cultural Heritage and New Technologies (CHNT 21). 2016. Vienna, Austria.

Wasserfarbe automatisch korrigieren

Die [automatisierte Korrektur der Unterwasseraufnahmen](#) ist ein entscheidender Schritt für die korrekte 3D-Modellerstellung von Unterwasserszenarien [1].

Veruda/Kroatien März 2016



[1] Block M., Gehmlich B., Hettmanczyk D.: "Automatic Underwater Image Enhancement using Improved Dark Channel Prior", In: Journal of Studies in Digital Heritage, Indiana University, Vol. 1, No. 2, pp. 566-589, 2017



Kommen wir jetzt zum finalen Projektteil ...!

Jetzt wollen wir alles kombinieren



Wasserdynamik



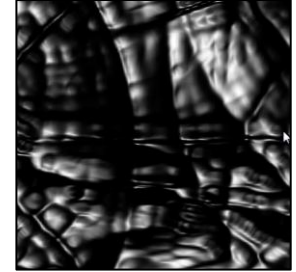
Refraktion



Schatten



Reflexion



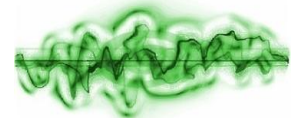
Kaustik



Bodentextur



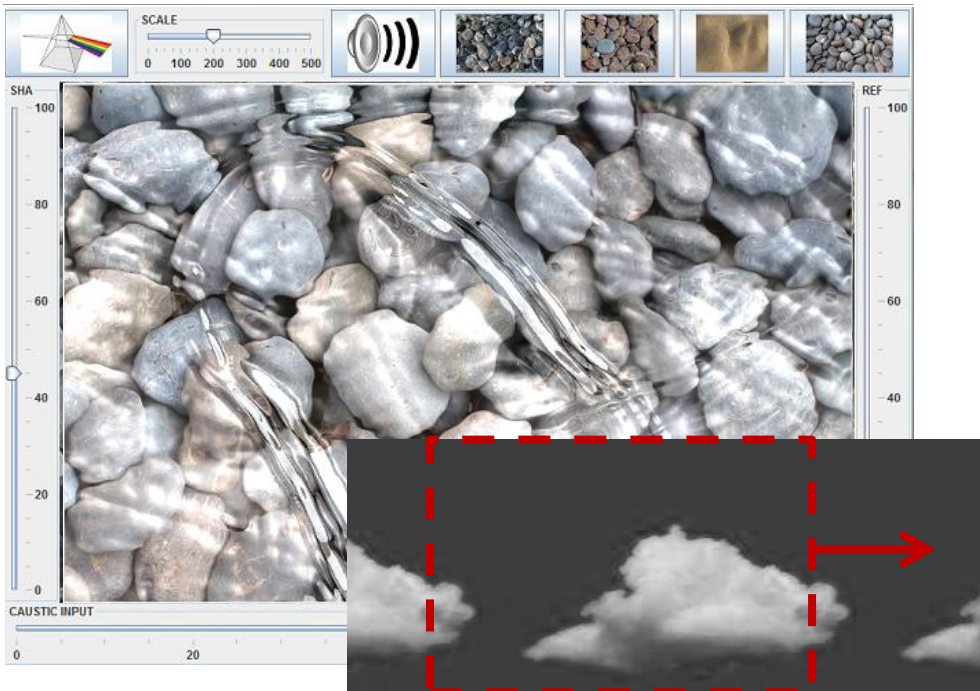
bewegtes
Wolkenbild



Plätschern

Projektziel – Stufe 7

In der siebenten Projektstufe kommen alle Effekte in einer experimentellen GUI zusammen. Die Bodentexturen sollen austauschbar sein und neben ein paar Soundeffekten kommen auch noch spiegelnde Wolken auf der Wasseroberfläche hinzu.



Projektstufe 7

- Experimentelle GUI
- Bodentexturen sollen austauschbar sein
- Effekte können parametrisiert eingebunden werden
- Ein zusätzlicher Wolkeneffekt wird sichtbar, wenn das Wasser nahezu still steht

Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation (Visualisierung) an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(s.x, 0))).x;
    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);
    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0-w.w) * vec4(1.0, 1.0, 1.0, 1.0) +
                     w.w * vec4(shadow, shadow, shadow, 1.0);

    float maxL      = 0.025;
    float weight     = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRefl  = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRefl    = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
}
}
```


Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(+s.x, 0))).x;
    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, +s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);
    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0 * w.w * vec4(1.0, 1.0, 1.0, 1.0) +
                      w.w * vec4(shadow, shadow, shadow, 1.0));

    float maxL      = 0.025;
    float weight     = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRef1  = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRef1   = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRef1 +
        w.z * wolkenRef1;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRef1 +
        w.z * wolkenRef1;
}
}
```

Gewichtete Effektkombinationen von außen steuerbar.

Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(s.x, 0))).x;
    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);

    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0-w.w) * vec4(1.0, 1.0, 1.0, 1.0) +
                      w.w * vec4(shadow, shadow, shadow, 1.0);

    float maxL      = 0.025;
    float weight    = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRefl = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRefl  = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
}
}
```

Gradientberechnung, Normale zur Wasseroberfläche,
Refraktion und Reflexion

Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(s.x, 0))).x;

    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, s.y))).y;

    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));

    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);

    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0-w.w) * vec4(1.0, 1.0, 1.0, 1.0) +
                     w.w * vec4(shadow, shadow, shadow, 1.0);

    float maxL      = 0.025;
    float weight     = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRefl  = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRefl    = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
}
}
```

Schattenberechnung

Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(s.x, 0))).x;
    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);
    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0-w.w) * vec4(1.0, 1.0, 1.0, 1.0) +
                      w.w * vec4(shadow, shadow, shadow, 1.0);

    float maxL      = 0.025;
    float weight     = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRefl  = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRefl    = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
}
}
```

Wolkenreflexion auf der Oberfläche

Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(s.x, 0))).x;
    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);
    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0-w.w) * vec4(1.0, 1.0, 1.0, 1.0) +
                      w.w * vec4(shadow, shadow, shadow, 1.0);

    float maxL      = 0.025;
    float weight     = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRefl  = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRefl    = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
}
}
```

Mondreflexion für die Spots

Implementierung Fragment-Shader Wasser

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Wassersimulation an:

```
// Wasserdynamik, Boden, Mond, Kaustik, Wolken
uniform sampler2D tex1, tex2, tex3, tex4, tex5;
uniform bool withRefraction;
uniform vec4 w; // w=(boden, kaustik, wolkenRefl, shadow/mondRefl)

uniform vec2 s;
uniform float scale, offSetCloud;

void main (void) {
    vec2 myPos      = gl_TexCoord[0].st;
    float g_x       = (texture2D(tex1, myPos + vec2(-s.x, 0)) -
                       texture2D(tex1, myPos + vec2(+s.x, 0))).x;
    float g_y       = (texture2D(tex1, myPos + vec2(0, -s.y)) -
                       texture2D(tex1, myPos + vec2(0, +s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x),
                                       vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    vec3 reflectV   = reflect(n_view, n_surface);
    float shadow    = 1.0 + 10.0 * (g_x-g_y);
    vec4 shadowCol  = (1.0-w.w) * vec4(1.0, 1.0, 1.0, 1.0) +
                      w.w * vec4(shadow, shadow, shadow, 1.0);

    float maxL      = 0.025;
    float weight     = (maxL - length(vec2(g_x, g_y))) / maxL;
    vec4 wolkenRefl  = vec4((weight*(texture2D(tex5,myPos+
        vec2(offSetCloud*s.x,0.0)+reflectV.xy))).rgb, 1.0);
    vec4 mondRefl    = texture2D(tex3, 5.0*reflectV.xy + vec2(0.5, 0.5));
```

```
if (withRefraction) {
    gl_FragColor =
        w.x * texture2D(tex2, myPos+s*scale*refractV.xy)*shadowCol +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
} else {
    gl_FragColor =
        w.x * shadowCol * texture2D(tex2, myPos) +
        w.y * texture2D(tex4, myPos+s*scale*refractV.xy*0.25) +
        w.w * mondRefl +
        w.z * wolkenRefl;
}
}
```

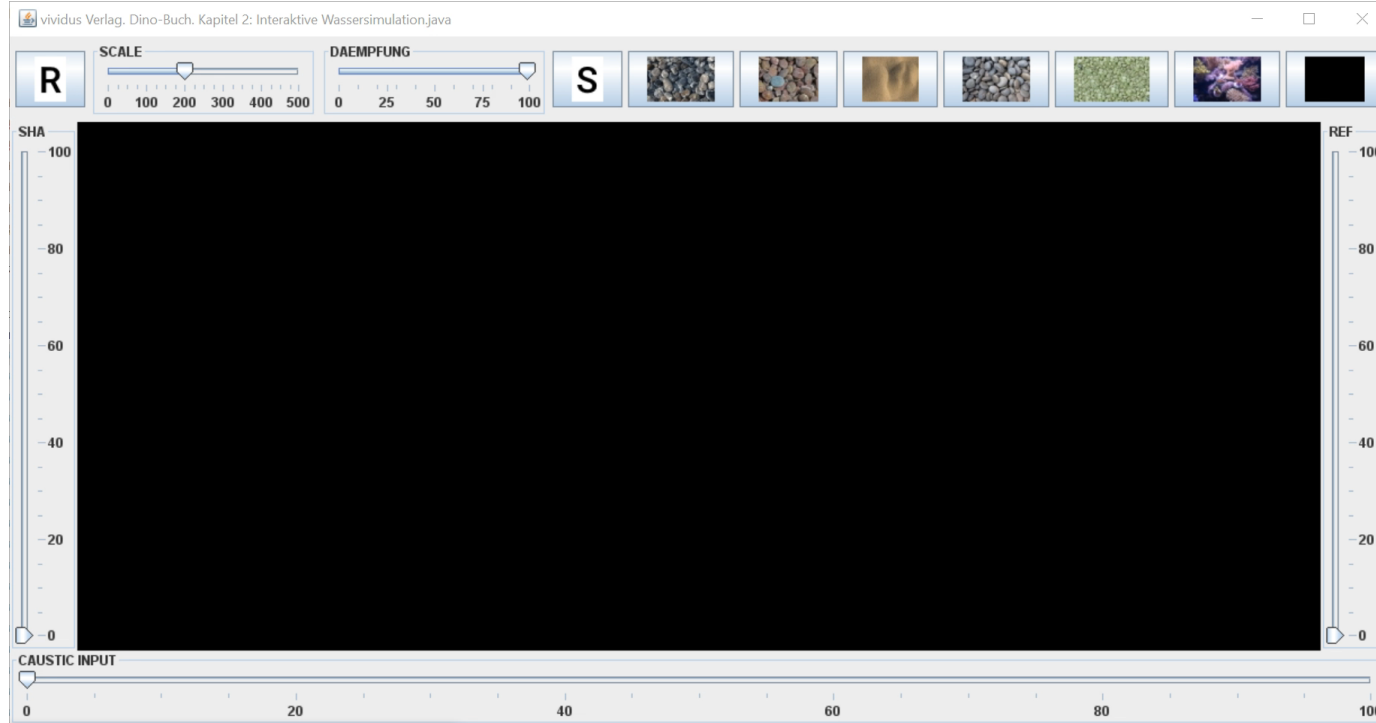
Gewichtete Effektkombinationen mit und ohne Refraktion von außen steuerbar



Endlich plätschern ...

Implementierung im Shader

Schauen wir uns das Ergebnis der experimentellen Wassersimulation an:



Video



Programmcode
kapitel02/Interaktive-
Wassersimulation.java

Prämierte Wassersimulation mit Fischeschwarm



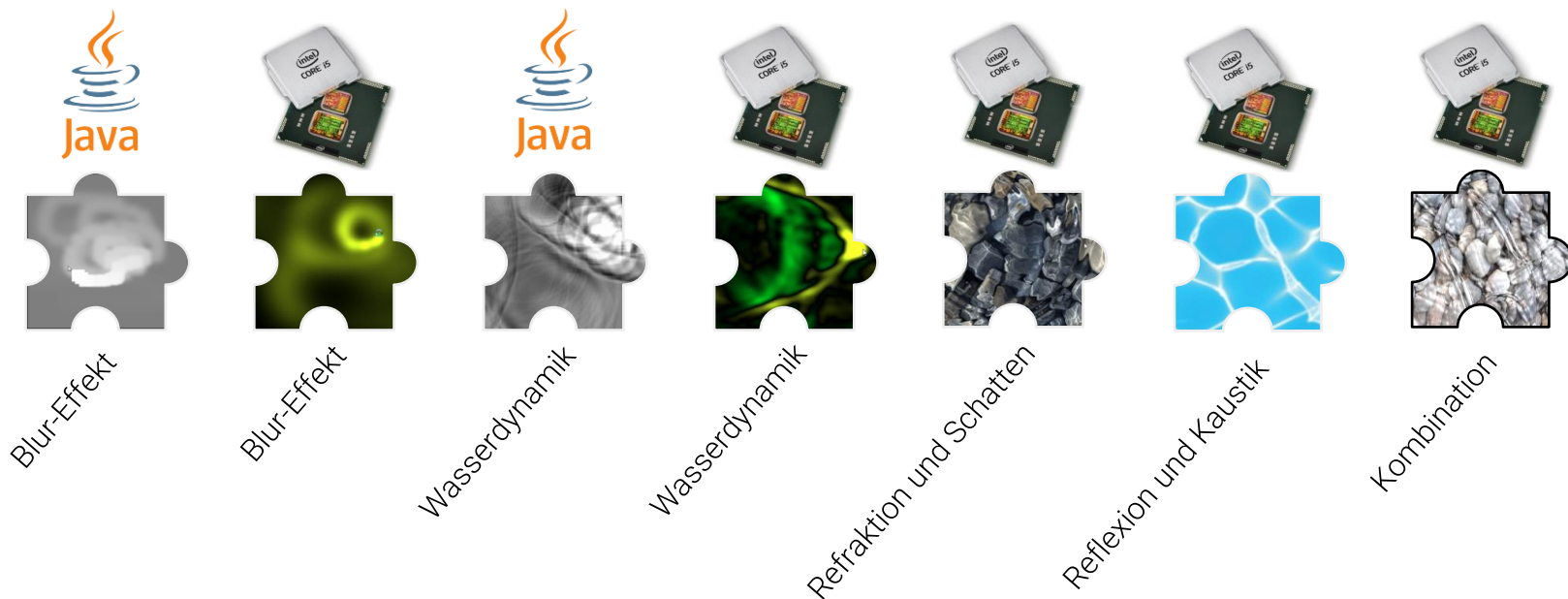
Eine phantastische Visualisierung in Kombination mit [Fischeschwärmen](#) hat q-bus 2009 auf der CeBIT vorgestellt.



Eine Kleinigkeit habe ich aber noch.

Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Thema „Für einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig werden wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.

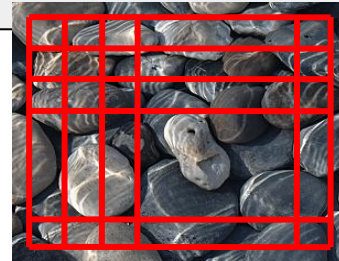
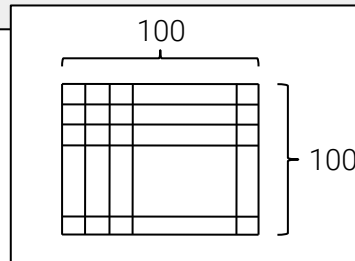


Implementierung einer Mesh-Struktur in OpenGL

Mit Hilfe des folgenden Programmfragments können wir die Textur GL_TEXTURE0 mit Linien horizontal und vertikal verknüpfen. Dazu konstruieren wir zwei Schleifen und erzeugen nebeneinanderliegende OpenGL-Primitive vom Typ GL_LINE_STRIP:

```
// kapitel02/GeometrieUndWasserdynamik.java
float leftC = -1.0f, rightC = 1.0f, topC = 1.0f, bottomC = - 1.0f;
float stepL = leftC, sizeL = (rightC-leftC) / 100.0f;
while (stepL <= rightC + sizeL) {
    glBegin(GL_LINE_STRIP);
    float startY = bottomC;
    while (startY <= topC + sizeL) {
        glMultiTexCoord2f(GL_TEXTURE0, (stepL-leftC)/(rightC-leftC),
                          (startY-leftC)/(rightC-leftC));
        glVertex3f(stepL, startY, 0.0f);
        startY += sizeL;
    }
    glEnd();
    stepL += sizeL;
}
```

```
float stepT = bottomC, sizeT = (topC-bottomC) / 100.0f;
while (stepT <= topC + sizeT) {
    glBegin(GL_LINE_STRIP);
    float startX = leftC;
    while (startX <= rightC + sizeT) {
        glMultiTexCoord2f(GL_TEXTURE0, (startX-bottomC)/(topC-bottomC),
                          (stepT-leftC)/(topC-bottomC));
        glVertex3f(startX, stepT, 0.0f);
        startX += sizeT;
    }
    glEnd();
    stepT += sizeT;
}
```

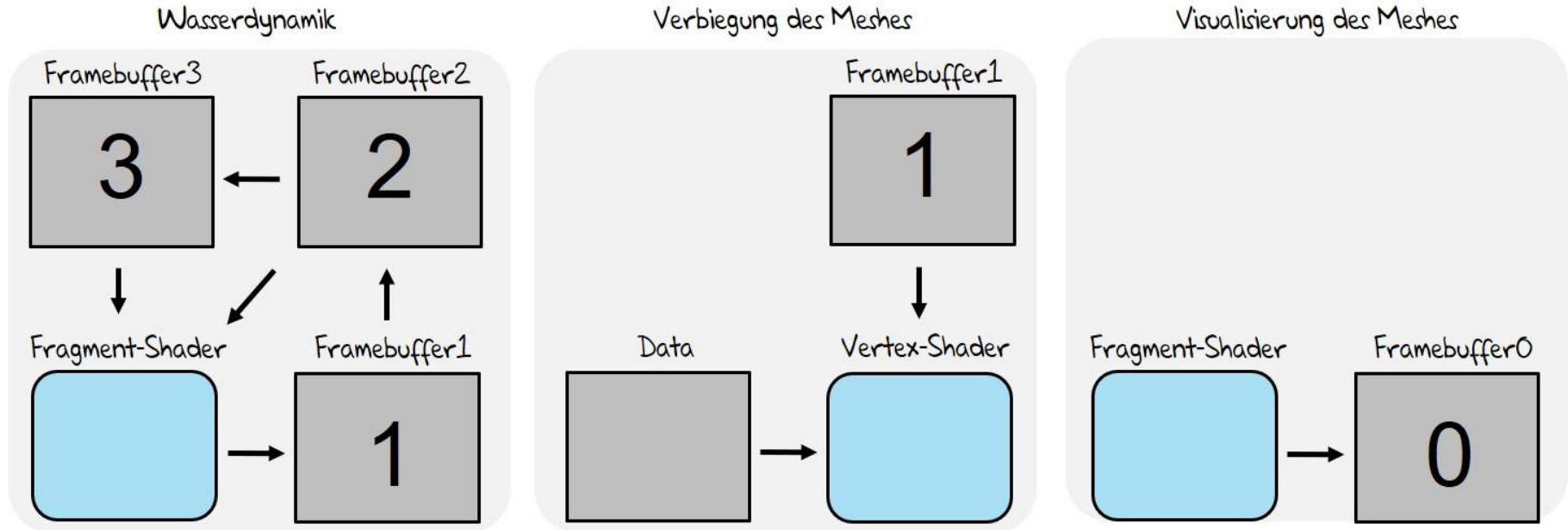




Schauen wir uns die Shaderstrategie an ...

Shader-Strategie für die Mesh-Verbiegung

Verkettung von Wasserdynamik (links), die Verschiebung aller Vertices (mittig) in Abhängigkeit zu der vorhandenen Wellendynamik und die Visualisierung des Results (rechts). Hier sehen wir das erste Mal eine Verkettung von drei Shadern hintereinander. Das Ergebnis der Wellendynamik wird hier allerdings direkt für die Manipulation der Geometrie genutzt.



Implementierung Vertex-Shader Mesh

Schauen wir uns jetzt den Vertex-Shadercode für die Mesh-Verbiegung an:

```
// kapitel02/GeometrieUndWasserdynamik.java
uniform sampler2D tex;
uniform vec2 s;
uniform float scale;
out vec4 frontColor;

void main (void) {
    vec2 myPos      = gl_MultiTexCoord0.st;
    float g_x       = (texture2D(tex, myPos + vec2(-s.x, 0)) - texture2D(tex, myPos + vec2(+s.x, 0))).x;
    float g_y       = (texture2D(tex, myPos + vec2(0, -s.y)) - texture2D(tex, myPos + vec2(0, +s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x), vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    frontColor      = vec4(1.0, 0.0, 0.0, 1.0);
    gl_Position     = (gl_ModelViewProjectionMatrix * gl_Vertex) + vec4(refractV.x, refractV.y, 0.0, 0.0)*s*scale;
}
```

Implementierung Vertex-Shader Mesh

Schauen wir uns jetzt den Vertex-Shadercode für die Mesh-Verbiegung an:

```
// kapitel02/GeometrieUndWasserdynamik.java
uniform sampler2D tex;
uniform vec2 s;
uniform float scale;
out vec4 frontColor;

void main (void) {
    vec2 myPos      = gl_MultiTexCoord0.st;
    float g_x       = (texture2D(tex, myPos + vec2(-s.x, 0)) - texture2D(tex, myPos + vec2(+s.x, 0))).x;
    float g_y       = (texture2D(tex, myPos + vec2(0, -s.y)) - texture2D(tex, myPos + vec2(0, +s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x), vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    frontColor      = vec4(1.0, 0.0, 0.0, 1.0);
    gl_Position     = (gl_ModelViewProjectionMatrix * gl_Vertex) + vec4(refractV.x, refractV.y, 0.0, 0.0)*s*scale;
}
```

Implementierung Vertex-Shader Mesh

Schauen wir uns jetzt den Vertex-Shadercode für die Mesh-Verbiegung an:

```
// kapitel02/GeometrieUndWasserdynamik.java
uniform sampler2D tex;
uniform vec2 s;
uniform float scale;
out vec4 frontColor;

void main (void) {
    vec2 myPos      = gl_MultiTexCoord0.st;
    float g_x       = (texture2D(tex, myPos + vec2(-s.x, 0)) - texture2D(tex, myPos + vec2(+s.x, 0))).x;
    float g_y       = (texture2D(tex, myPos + vec2(0, -s.y)) - texture2D(tex, myPos + vec2(0, +s.y))).y;
    vec3 n_surface  = normalize(cross(vec3(1.0, 0.0, g_x), vec3(0.0, 1.0, g_y)));
    vec3 n_view     = vec3(0.0, 0.0, 1.0);
    vec3 refractV   = refract(n_view, n_surface, 1.33);
    frontColor      = vec4(1.0, 0.0, 0.0, 1.0);
    gl_Position     = (gl_ModelViewProjectionMatrix * gl_Vertex) + vec4(refractV.x, refractV.y, 0.0, 0.0)*s*scale;
}
```



Aber wo bleibt hier der Fragment-Shader?

Implementierung Fragment-Shader Mesh

Schauen wir uns jetzt den kurzen Fragment-Shadercode für die [Mesh-Verbiegung](#) an:

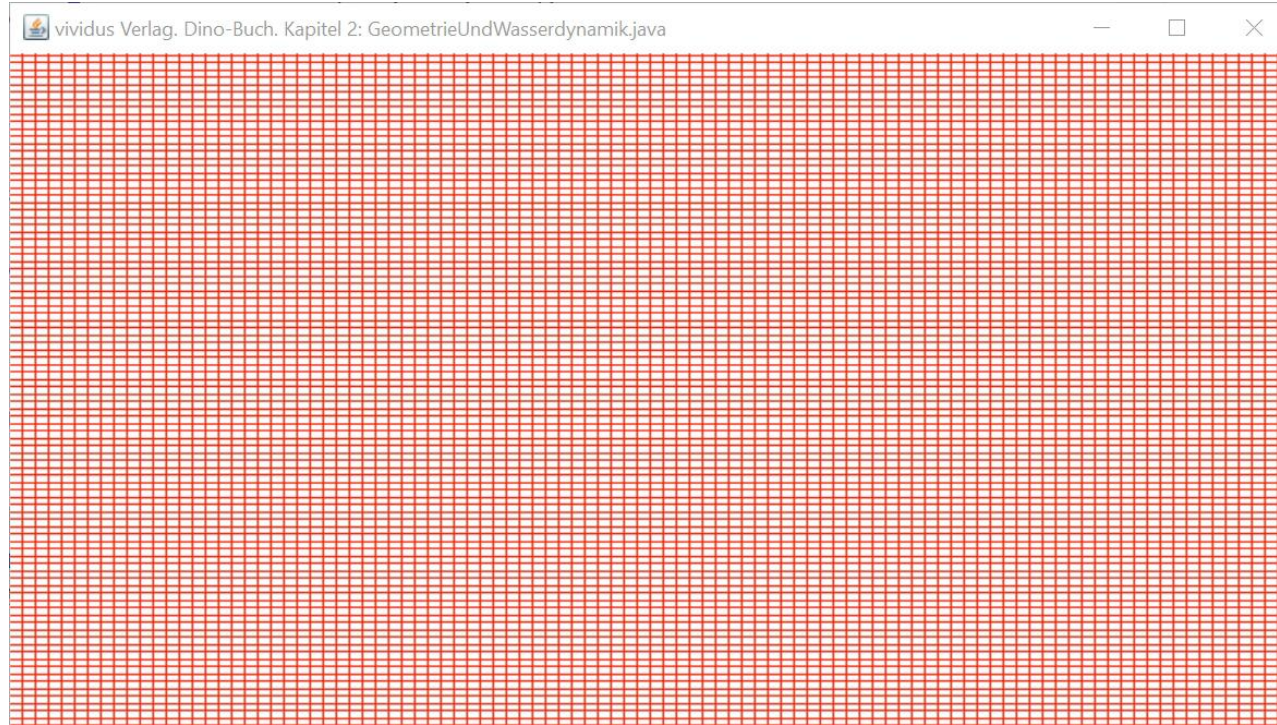
```
in vec4 frontColor;

void main (void) {
    gl_FragColor = frontColor;
}
```



Implementierung im Shader

Schauen wir uns das Ergebnis der Mesh-Verbiegung an:



Video



Programmcode

kapitel02/GeometrieUnd-
Wasserdynamik.java

Entwicklung einer experimentellen Wassersimulation

Key Takeaways

- Wasser entsteht visuell aus der Kombination vieler Einzeleffekte
- Reflexion, Refraktion und Kaustik prägen Wasseroberflächen
- Gradienten liefern Normalenvektoren für Lichtberechnungen
- Shader berechnen Wassereffekte effizient auf der GPU
- Effektgewichtungen ermöglichen flexible Wassersimulationen
- Fragment-Shader verändern Pixel und Oberflächenwirkung
- Vertex-Shader deformieren zusätzlich die Geometrie
- Wasserdynamik kann für Optik und Mesh-Verbiegung genutzt werden
- Mehrere Shader lassen sich zu Renderpipelines kombinieren
- Interaktive Eingaben erzeugen dynamische Wasserreaktionen



Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Listen Sie noch einmal alle Effekte auf, die für die Kombination der Wassersimulation implementiert wurden.
- (2) Nutzen Sie das warme Wetter, gehen Sie draußen schwimmen und baden. Welche weiteren Wassereffekte können Sie bei der aktiven Outdoor-Analyse identifizieren?

Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] Erweitern Sie die bestehende Wassersimulation um einen zusätzlichen visuellen Effekt Ihrer Wahl. Der Effekt soll parametrisiert steuerbar sein. Die Integration soll innerhalb der bestehenden Shaderpipeline erfolgen. Mögliche Erweiterungen:

- farbabhängige Tiefensimulation
- Regen- oder Tropfeneffekte
- zusätzliche Lichtreflexionen
- animierte Unterwasserobjekte
- bewegte Strömungen
- verbesserte Kaustiken
- Nebel- oder Schaumdarstellung
- interaktive Maus- oder Touch-Eingaben

Die mathematische oder physikalische Grundlage soll kurz dokumentiert werden. Analysieren Sie, ob der neue Effekt besser im Vertex-Shader oder im Fragment-Shader implementiert werden sollte und begründen Sie Ihre Entscheidung.



Vielen Dank für die Aufmerksamkeit